nwctl User Manual

nwctl User Manual

Nuway Autoware Container Manager — Multi-user Autoware Docker testing and development toolkit

---

Quick Command Reference

User Management

Command	Description
`nwctl register <name> --src <path>`	Register a new user with source directory
`nwctl register <name> --src <path> --domain-id <0-232>`	Register with a specific available ROS domain ID
`nwctl update <name> --src <path>`	Update user's source path
`nwctl unregister <name>`	Remove user and delete build/install/log workspace
`nwctl unregister <name> --keep-workspace`	Remove user (explicitly keep workspace)
`nwctl list`	List all registered users and their DOMAIN_IDs

Run Modes

Command	Description
`nwctl <name> planning-sim`	Launch planning simulation (uses prebuilt image)
`nwctl <name> rosbag-replay`	Launch rosbag replay (uses prebuilt image)
`nwctl <name> shell`	Enter development shell (mounts user source)
`nwctl <name> stop`	Stop all running containers for a user
`nwctl <name> clean`	Clean user's build cache

Status & Maintenance

Command	Description
`nwctl status`	Show all running aw-* containers
`nwctl disk`	Show disk usage per user (workspace)
`nwctl cleanup`	Remove orphan containers and temporary files
`nwctl cleanup --dry-run`	Preview what cleanup would remove (no changes)
`nwctl check-env [mode]`	Pre-flight check (Docker/GPU/image/display/data/resources)
`nwctl version`	Show version number
`nwctl pull`	Pull or update the configured container image
`nwctl pull --profile cpu`	Pull the no-GPU Humble development image
`nwctl completion <bash|zsh>`	Print completion setup for a source checkout
`nwctl -h`	Show help

CPU and NVIDIA Profiles

The default `auto` profile selects `nvidia` only when both an NVIDIA GPU and the Docker NVIDIA runtime are available; otherwise it selects `cpu`.

Profile	Image	Runtime
`cpu`	`ghcr.io/autowarefoundation/autoware:universe-devel-humble`	Software rendering, no NVIDIA runtime
`nvidia`	`ghcr.io/autowarefoundation/autoware:universe-devel-cuda`	`--runtime=nvidia`

```
nwctl check-env shell --profile cpu
nwctl <name> shell --profile cpu
nwctl <name> planning-sim --profile cpu
```

Set `NWCTL_PROFILE=cpu` to make CPU mode the default for a host/session.

Run Modes — Details

1. planning-sim — Planning Simulation

```
nwctl <name> planning-sim
```

- Uses `/opt/autoware` inside the image (prebuilt) — **no local compilation required**
- Opens an rviz2 window via X11 forwarding automatically

Steps:

1. Wait for the rviz2 window to appear (~30 seconds)
2. Click **2D Pose Estimate** in the toolbar, then click and drag on the map to set the initial pose
3. Click **2D Goal Pose** to set the destination
4. Click the buttons in the **AutowareStatePanel** to start autonomous driving

---

2. rosbag-replay — Bag File Replay

```
nwctl <name> rosbag-replay
nwctl <name> rosbag-replay --bag <subdirectory> --rate 0.5
```

- Uses the prebuilt version inside the image
- Starts Autoware and runs `ros2 bag play` after the initialization delay
- `--bag` selects a subdirectory below `--rosbag-path`; `--rate` controls speed
- The CPU profile disables the CUDA/ML-model perception pipeline and waits 60 seconds before playback; map loading, sensor decoding, localization, and rviz remain enabled
- Set `NWCTL_ROSBAG_START_DELAY` to override the automatic playback delay
- In the rviz **Views** panel, set **Target Frame** to `base_link` to follow the vehicle

---

3. shell — Development Mode

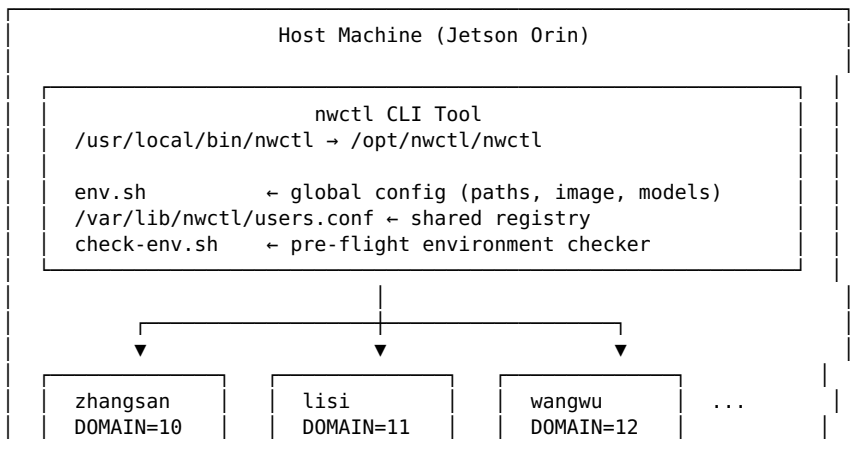
```
nwctl <name> shell
```

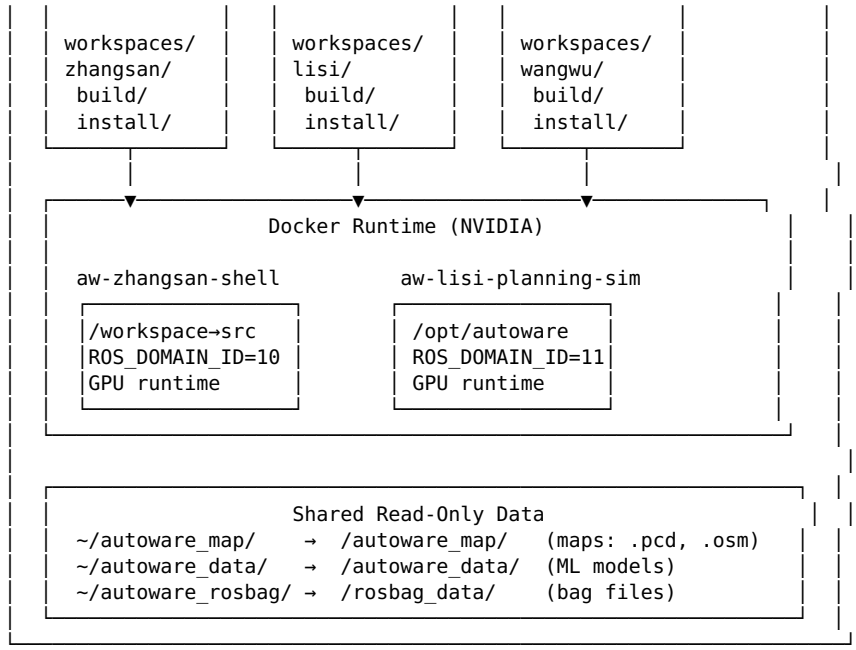
- Mounts the user's source directory to `/workspace/src` inside the container
- Overlays `/opt/autoware` (prebuilt) as the base dependency layer

Common operations inside the container:

```
# Incremental build — single package (most common)
colcon build --packages-select <pkg> --symlink-install

# Build a package with all its dependencies
colcon build --packages-up-to <pkg> --symlink-install

# Full build (first time only — takes a while)
colcon build --symlink-install --cmake-args -DCMAKE_BUILD_TYPE=Release

# Re-source and test with planning simulation
source /workspace/install/setup.bash
ros2 launch autoware_launch planning_simulator.launch.xml \
    map_path:=/autoware_map/sample-map-rosbag \
    vehicle_model:=sample_vehicle \
    sensor_model:=sample_sensor_kit
```

Architecture Diagram





Container Mode Comparison

Mode	Source	setup.bash	Use Case
`planning-sim`	Prebuilt in image	`/opt/autoware/setup.bash`	Testing, demo, validation
`rosbag-replay`	Prebuilt in image	`/opt/autoware/setup.bash`	Data replay and analysis
`shell`	User-mounted source	`/workspace/install/setup.bash`	Development, debugging, algorithm changes

---

Isolation Mechanisms

Resource	Method	Details
ROS topics	`ROS_DOMAIN_ID`	Auto-assigned per user, starting at 10, skipping reserved IDs (5)
Source code	Separate git clones	Each user manages their own repository and branches
Build artifacts	Separate mounts	`/var/lib/nwctl/workspaces/<user>/build`, `install`, `log`
Container names	`aw-<user>-<mode>`	Prevents conflicts, easy to identify
Registry writes	`flock` (10s timeout)	`users.conf.lock` prevents concurrent write conflicts
GPU	Shared	NVIDIA runtime supports multiple containers simultaneously

---

Typical Workflow

Admin (one-time setup)

```
# Pull the Docker image
docker pull ghcr.io/autowarefoundation/autoware:universe-devel-cuda

# Install nwctl
cd ~/autoware/nwctl
sudo ./install.sh
# Bash/Zsh completion is installed too; start a new shell and press Tab.
```

Each Team Member

```
# 1. Clone your own copy of the code
cd ~
git clone https://github.com/autowarefoundation/autoware.git myname_aw
cd myname_aw
vcs import src < repositories/autoware.repos

# 2. Register
nwctl register myname --src ~/myname_aw/src
```

```
# 3. Pre-flight check
nwctl check-env

# 4. Use as needed
nwctl myname planning-sim     # testing
nwctl myname rosbag-replay    # replay
nwctl myname shell            # development
```

CI/CD Pipeline

```
Push to dev/main branch
        │
        ▼
  ci.yaml (triggered automatically)
  ├── shellcheck static analysis
  └── basic validation (executable, symlink, version, help, install/uninstall)

Push tag v*
        │
        ▼
  release.yaml (triggered automatically)
  ├── reuse ci.yaml
  ├── verify tag matches NWCTL_VERSION in script
  ├── create nwctl-X.Y.Z.tar.gz
  ├── auto-generate changelog
  └── publish GitHub Release (personal + org repos)
```

Remote Repositories:

Alias	URL	Purpose
`origin`	`git@github.com:LiZheng1997/nwctl.git`	Personal repo
`org`	`git@github.com:uwa-rev/nwctl.git`	Organization repo

FAQ

Q: rviz2 shows a black screen or crashes

```
xhost +local:docker
echo $DISPLAY   # Should be :0 or :1
```

Q: GPU error "could not select device driver"

```
docker run --rm --runtime=nvidia nvidia/cuda:12.0.0-base-ubuntu22.04 nvidia-smi
# If this fails, reinstall nvidia-container-toolkit and restart Docker
```

Q: Multiple users' ROS topics interfere with each other

Each user is automatically assigned a unique `ROS_DOMAIN_ID`. Isolation is by design — no manual action needed.

Q: colcon build can't find dependencies

```
source /opt/autoware/setup.bash   # source prebuilt layer first
colcon build --packages-select <pkg> --symlink-install
```

Q: Permission error on pointcloud map

```
sudo chmod 644 ~/autoware_map/sample-map-rosbag/pointcloud_map.pcd
# Or run the automated check:
nwctl check-env
```

Q: Are build artifacts preserved after container exit?

Yes. In an installed deployment, build artifacts live in `/var/lib/nwctl/workspaces/<user>/` and persist across container restarts.

Q: How do I switch branches for testing?

Manage your code on the host:

```
cd ~/myname_aw/src/universe/autoware_universe
git checkout feature/new-branch
# Then enter the dev shell and rebuild
nwctl myname shell
```

---

nwctl v0.2.1 — Nuway Autoware Team